UNIVERSITÀ DEGLI STUDI DI SALERNO



DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE ED ELETTRICA E MATEMATICA APPLICATA

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

RELAZIONE DI

SISTEMI EMBEDDED

Docente:
Prof. Percannella Gennaro
Prof. Greco Luca

Membri:
Luongo Leonardo – 0622702606
Monda Francesco – 0622702620

Gruppo:
38

Anno Accademico 2024/2025

INDICE

CAPITOLO I - INTRODUZIONE *2*

1.1. Funzionalità generale e configurazione del dispositivo **2**

1.2. Attivazione e comportamento in stato attivo o disattivo **3**

1.3. Visualizzazione della temperatura con LED RGB **3**

1.4. Controllo dell'illuminazione **3**

1.5. Specifica dei Requisiti di Sistema **4**

1.6. Collegamenti hardware: setup dei PIN **5**

1.7. PuTTY **6**

CAPITOLO 2 - SOLUZIONE *8*

2.1. Architettura Software **8**

2.2. Gestione degli stati in Parallelo **9**

2.3. LightManagment **11**

2.4. VBlindsManagement **13**

2.5. NotifyUART **15**

2.6. HeatingManagement **16**

2.7. BlinkingManagement **18**

CAPITOLO III – ESECUZIONE PROGETTO *19*

INDICE DEI CODICI, DELLE TABELLE E DELLE FIGURE *21*

CAPITOLO I - INTRODUZIONE

Il progetto che abbiamo sviluppato ha l'obiettivo di realizzare un **sistema domotico embedded**. Il sistema è in grado di controllare tre aspetti fondamentali dell'ambiente domestico: la **regolazione delle veneziane**, la **gestione dell'illuminazione** e la **visualizzazione della temperatura impostata**. Il cuore del sistema è un microcontrollore che dialoga con l'utente attraverso un'interfaccia UART e una serie di pulsanti fisici.

1.1. Funzionalità generale e configurazione del dispositivo

Al primo avvio, il dispositivo entra automaticamente in **modalità di configurazione**, poiché non dispone di alcuna impostazione predefinita. In questa fase, tramite collegamento **UART**, viene presentato un **menù interattivo** che guida l'utente nella selezione delle funzionalità da impostare. La configurazione può avvenire solo se il sistema è stato prima attivato.

Primo Accesso alla Configurazione

Alla prima entrata nello stato di configurazione, **tutti i parametri devono essere impostati almeno una volta** prima di poter uscire da questo stato. È sempre possibile premere il pulsante **CONFIG** per accedere alla configurazione, ma l'uscita sarà abilitata solo quando ogni valore è stato modificato.

Per agevolare l'utente, il sistema visualizza la seguente stringa di stato:

```
Temp: x Angle: x Timer: x Relay: x
```

Code 1 - Stato di Configurazione (valori non settati)

Dove `x` indica un parametro **non ancora modificato** e `o` indica un parametro **già modificato almeno una volta**.

Quando tutti i parametri risultano modificati, la riga diventa:

```
Temp: o Angle: o Timer: o Relay: o
```

Code 2 - Stato di Configurazione (valori settati)

Solo in questo stato sarà possibile uscire dalla configurazione.

Accessi Successivi

Dalla seconda volta in poi, per uscire dallo stato di configurazione è sufficiente modificare **almeno un parametro** rispetto alla configurazione salvata.

Durante la configurazione, l'utente può:

- regolare l'angolo di apertura delle veneziane tra 0° e 180°, con incrementi di 20°;
- accendere o spegnere le luci, o impostare una durata temporizzata di accensione compresa tra 5 secondi e 30 secondi (con 0 secondi che disattiva questa modalità);
- selezionare una temperatura tra 16°C e 30°C, con passi di 1°C.

Ogni valore inserito viene validato dal sistema: se corretto, la configurazione viene accettata e salvata; in caso contrario, il sistema rifiuta il valore e mantiene l'ultima configurazione valida (se presente).

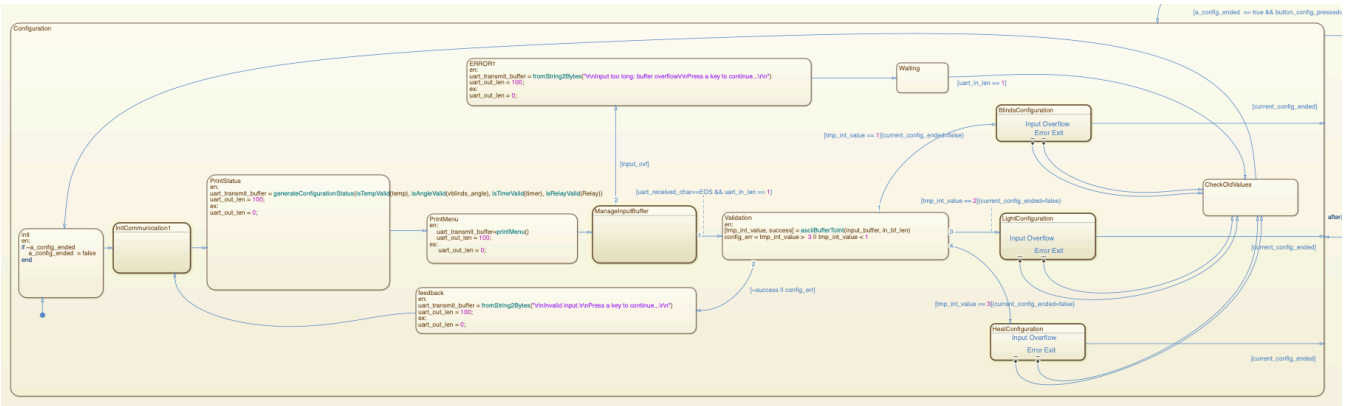


Figura 1 - Stato Configuration

1.2. Attivazione e comportamento in stato attivo o disattivo

Il sistema all'avvio si presenta inattivo (`OFF_STATE`): tutte le uscite sono disabilitate, le luci sono spente, le veneziane sono chiuse e la temperatura è al minimo. L'attivazione del dispositivo avviene premendo il pulsante ATTIVA. Quando attivo (`Active_State`), il sistema applica tutte le configurazioni salvate e segnala la sua operatività tramite il LED utente, che lampeggia a una frequenza di 2Hz.
Durante il funzionamento normale (cioè quando non si è in modalità di configurazione), ogni 10 secondi viene inviato via UART lo stato corrente di tutte le funzionalità.

1.3. Visualizzazione della temperatura con LED RGB

Uno degli aspetti più intuitivi del sistema è la visualizzazione della temperatura attraverso un LED RGB, che cambia colore a seconda del valore impostato. È una forma di feedback immediato che consente di percepire la temperatura selezionata senza dover consultare valori numerici. Il comportamento del LED è così definito:

TEMPERATURE	COLORE
tra 16°C e 18°C	blu
tra 19°C e 21°C	viola
tra 22°C e 24°C	giallo
tra 25°C e 27°C	arancione
tra 28°C e 30°C	rosso

Tabella 1 – Visualizzazione della Temperatura con LED RGB

1.4. Controllo dell'illuminazione

La gestione delle luci è affidata a un relè che ne consente l'accensione o lo spegnimento. Il sistema consente di agire sulle luci in due modi: attraverso l'interfaccia UART oppure mediante i pulsanti LUCI e LUCI TMP.
Il pulsante LUCI permette di alternare lo stato della luce: se spenta, si accende; se accesa, si spegne; se è attiva una temporizzazione, può essere disattivata anticipatamente. Il pulsante LUCI TMP, invece, avvia l'accensione temporizzata, secondo la durata impostata in configurazione. Questo pulsante ha effetto solo se le luci non sono già in fase di temporizzazione.

1.5. Specifica dei Requisiti di Sistema

Il sistema deve garantire:

ID REQ	REQUISITO UTENTE	STATO COINVOLTO
R1	Regolazione apertura veneziane (0° – 180°, step 20°)	`VBlindsManagement`
R2	Accensione/spegnimento luci tramite LUCI o UART	`LightManagement`
R3	Accensione temporizzata delle luci (5s – 30s)	`LightManagement`
R4	Feedback temperatura via LED RGB (16°C – 30°C)	`HeatingManagement`
R5	Trasmissione stato via UART ogni 10s quando attivo	`NotifyUART`
R6	Indicazione di stato attivo tramite LED lampeggiante a 2Hz	`BlinkingManagement`
R7	Modalità configurazione accessibile solo da stato attivo	`Configuration`
R8	Rigetto valori errati in configurazione	`Validation`
R9	Stato iniziale disabilitato con output spenti	`OFF_STATE`

Tabella 2 - Analisi dei Requisiti

1.6. Collegamenti hardware: setup dei PIN

Per realizzare le funzionalità descritte, ogni componente hardware è stato connesso a specifici pin del microcontrollore. La tabella seguente riassume in modo chiaro e ordinato la disposizione prevista per i collegamenti:

DISPOSITIVO	PIN	NOTE / FUNZIONALITÀ
LED RGB - rosso	PA6	TIM3 PWM CH1
LED RGB - verde	PC7	TIM3 PWM CH2
LED RGB - blu	PC8	TIM3 PWM CH3
LED utente	PA5	Indicatore di stato
Controllo servomotore	PB6	TIM4 PWM CH1
Controllo relè (luci)	PC2	Gestione accensione
Pulsante CONFIG	PB0	Ingresso/uscita config
Pulsante ATTIVA	PC13	User button
Pulsante LUCI	PB1	Accensione/Spegnimento
Pulsante LUCI TMP	PA15	Accensione temporizzata
UART RX	PA3	LPUART RX
UART TX	PA2	LPUART TX

Tabella 3 – setup dei PIN

1.7. PuTTY

Per la comunicazione tra l'utente e il microcontrollore attraverso l'interfaccia seriale UART, è stato utilizzato PuTTY, un emulatore di terminale open-source. Questo strumento consente di stabilire una connessione seriale per inviare comandi di configurazione al dispositivo e ricevere i messaggi di stato.
Per interagire correttamente con il sistema, PuTTY deve essere configurato con i seguenti parametri:

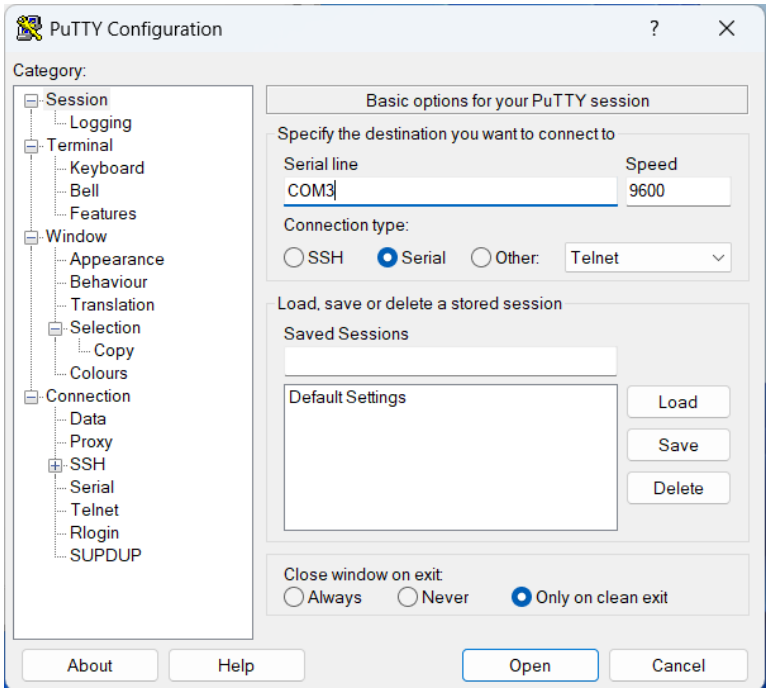


Figura 2 - Configurazione PuTTY

Una volta stabilita la connessione, PuTTY apre una finestra di terminale che funge da console per dialogare con il sistema embedded, visualizzando il menù di configurazione e i dati inviati dal dispositivo.

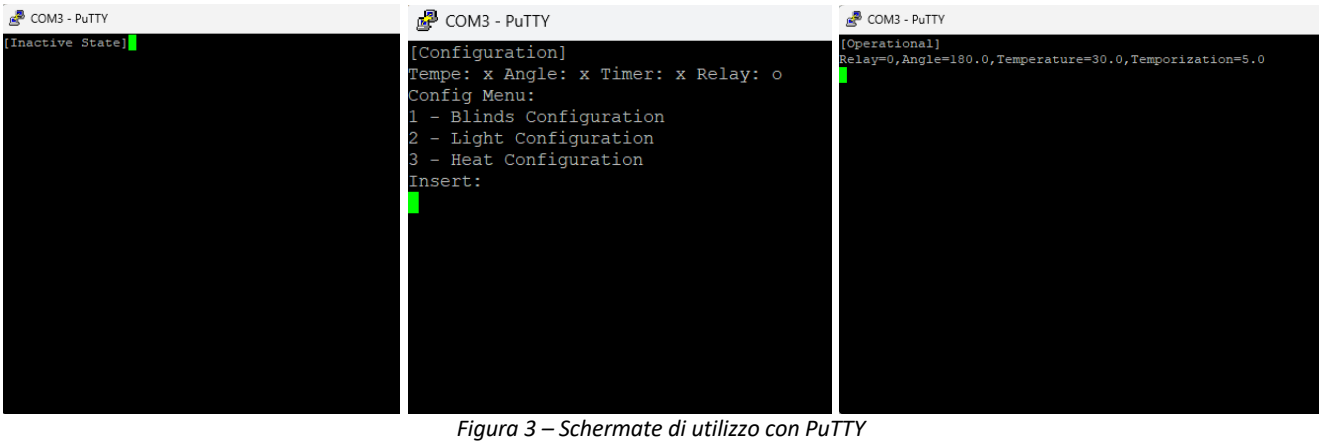


Figura 3 – Schermate di utilizzo con PuTTY

Nel nostro progetto, in modalità configurazione è possibile inserire **solo:**

- **valori numerici a virgola mobile maggiori o uguali a 0**, dove per virgola si intende il carattere punto (Es: “`3.0`”);
- **caratteri speciali** come “backspace” e “invio”.

Gli altri caratteri non verranno mostrati a schermo. Il limite massimo di caratteri inseribili è pari a 20, qualora si superasse il limite, il sistema fornirà un messaggio di errore dettagliato.
Il sistema è Fault-Tolerant ad input non validi, fornendo un messaggio di errore.
Nel caso in cui si invia una sequenza escape multi-carattere per esempio “CANC”, formato da:

- `27` (il carattere ESC)
- `91` (il carattere [)
- `51` (il carattere '3')
- `126` (il carattere ~)

in console sarà visibile il carattere `3`.

CAPITOLO 2 - SOLUZIONE

Per affrontare la realizzazione del sistema embedded si è adottato un approccio top-down, partendo da una visione ad alto livello delle funzionalità richieste e procedendo per raffinamenti successivi fino alla definizione delle componenti software e hardware di basso livello. Questo approccio ha permesso di mantenere una visione chiara della struttura complessiva del sistema durante tutto lo sviluppo, facilitando anche la verifica dei requisiti.

2.1. Architettura Software

Dal punto di vista architetturale, è stata adottata una suddivisione modulare delle componenti software, separando nettamente la logica di configurazione, il controllo dell'hardware, la gestione dello stato interno e il trattamento degli input (sia da UART sia da pulsanti). Ogni modulo è stato progettato per essere il più autonomo possibile.
La comunicazione tra i moduli avviene tramite strutture dati condivise, che rappresentano lo stato corrente del sistema, aggiornato dinamicamente nel `main loop` principale.

Per implementare la logica di controllo del sistema è stata sviluppata una macchina a stati finiti (FSM) utilizzando l'ambiente Stateflow di MATLAB. La *Figura 1* sottostante mostra l'architettura completa del modello, che rappresenta il "cervello" del nostro sistema. Il diagramma evidenzia la scomposizione gerarchica in macro-stati che gestiscono le diverse modalità operative, dalla configurazione iniziale all'esecuzione delle funzionalità principali. Questa vista d'insieme funge da introduzione all'analisi dettagliata di ogni stato, che sarà affrontata nelle sezioni successive.

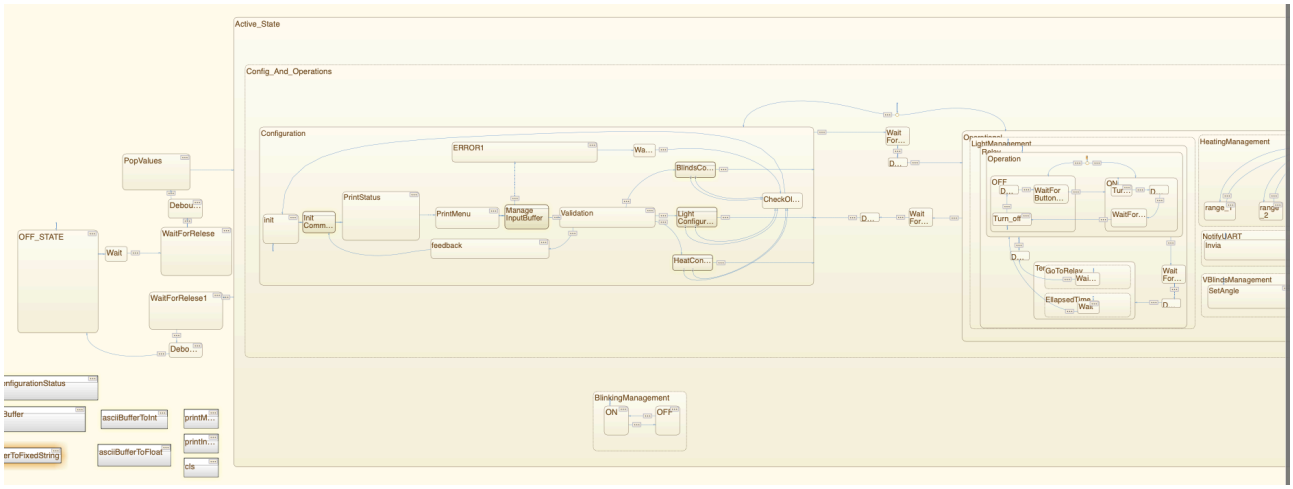


Figura 4 - Modello Stateflow della macchina a stati finiti per la gestione del sistema

2.2. Gestione degli stati in Parallelo

All'interno dello **stato Attivo**, sono presenti **sottostati paralleli** che operano contemporaneamente per garantire il corretto funzionamento del sistema.
In particolare, lo stato `Config_And_Operations` viene eseguito in parallelo con `BlinkingManagement`.
Questo approccio è stato scelto poiché il lampeggio del LED (Blinking) deve essere attivo ogni volta che la scheda si trova nello stato **Attivo**, indipendentemente dal fatto che sia in fase di **configurazione** o in modalità **operativa**.

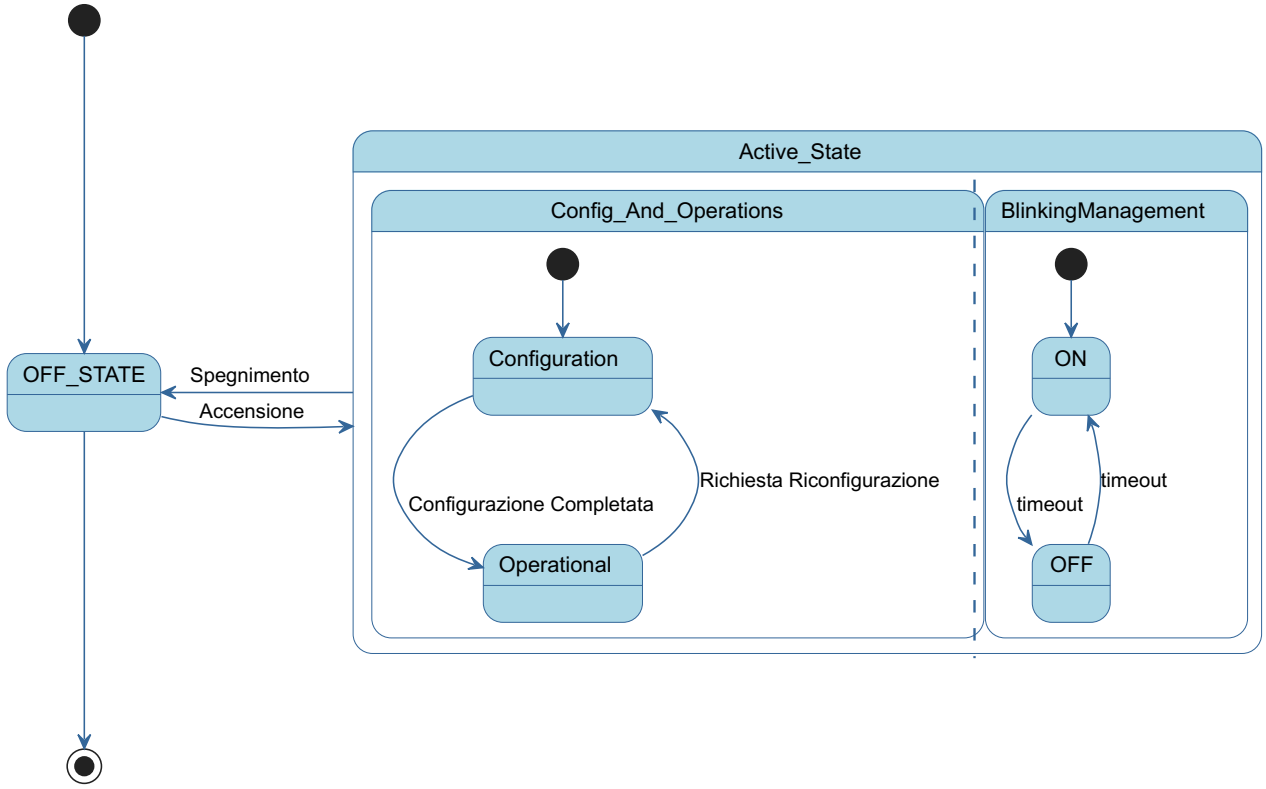


Figura 5 - Concorrenza nello stato Attivo

All'interno dello stato `Operational`, sono gestiti in parallelo i seguenti sottostati:

- **LightManagement**,
- **VBlindsManagement**,
- **NotifyUART**,
- **HeatingManagement**.

Questi stati devono rimanere **attivi simultaneamente** per poter rispondere in modo reattivo agli **stimoli esterni** (input dell'utente, timer, variazioni ambientali).

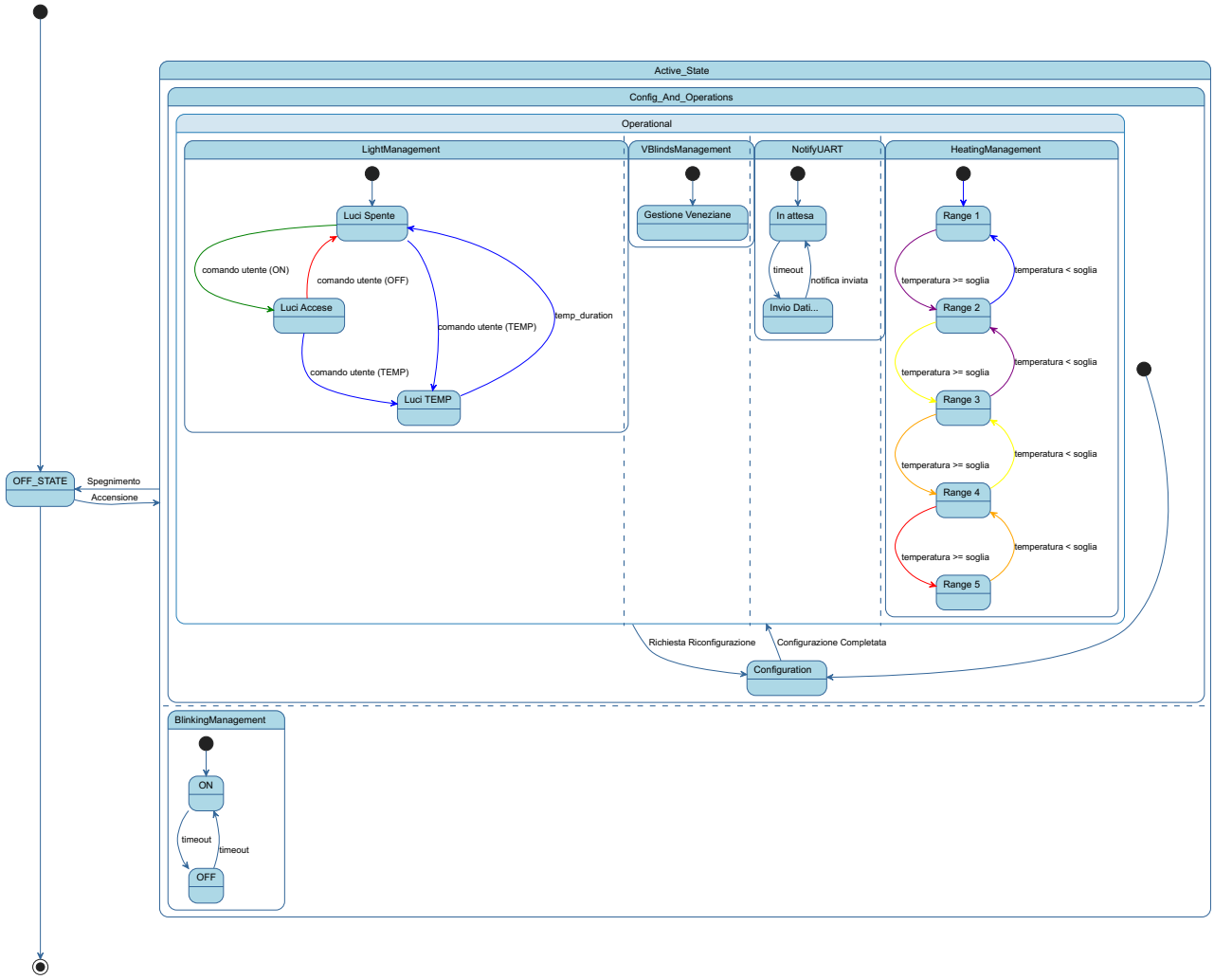


Figura 6 - Concorrenza nello stato Operational

2.3. LightManagment

Il modulo `LightManagement` controlla l'illuminazione, permettendo all'utente di accendere e spegnere le luci tramite il pulsante LUCI e di attivarle in modalità temporizzata tramite il pulsante LUCI TMP. Il controllo fisico delle luci avviene tramite un relè, il cui stato (`Relay=ON` o `Relay=OFF`) è gestito da questo modulo. Una caratteristica importante è la gestione del debounce dei pulsanti per evitare false attivazioni dovute al rimbalzo meccanico dei contatti.
Il modulo può essere concettualmente suddiviso in due principali sotto-macchine a stati parallele o modalità operative che interagiscono:

- ◊ **"Operation":** Gestisce l'accensione e lo spegnimento standard.
- ◊ **"Temp":** Gestisce l'accensione per un periodo limitato.

All'ingresso in questo modulo, le luci sono inizialmente OFF. Il relè è disattivato (`Relay=OFF`).

"Operation" (Gestione Luci Normali)

Stato OFF (Luci Spente)

Il sistema è in attesa della pressione del pulsante LUCI (`button_light_pressed`).
Quando `button_light_pressed` diventa OFF, si entra in uno stato transitorio `WaitForButtonPressing` e poi in uno stato `Debounce`.
Dopo la rilevazione iniziale della pressione, il sistema attende un breve periodo (`debounce`) per ignorare i rimbalzi del pulsante. Durante questo periodo, ulteriori transizioni del segnale del pulsante vengono ignorate. Lo scopo è assicurarsi che la pressione sia stabile e intenzionale.
Dopo aver atteso il *rilascio* e poi un `debounce` sul rilascio, il sistema transita allo stato ON.

Stato ON (Luci Accese)

Il sistema è in attesa della pressione del pulsante LUCI (`button_light_pressed`).
Quando `button_light_pressed` diventa ON, si entra nuovamente in uno stato transitorio `WaitForButtonPressing` e poi `Debounce`. Successivamente il sistema transita nuovamente allo stato OFF.

"Temp" (Gestione Luci Temporizzate)

Questa modalità può essere attivata sia dallo stato OFF sia dallo stato ON della sotto-macchina **"Operation"** quando viene premuto il pulsante LUCI TMP (`button_temp_pressed == ON`).
Anche qui, la pressione del pulsante LUCI TMP dovrebbe passare attraverso una logica di attesa rilascio e `debounce` per confermare l'intenzione.

Stato Temp:

All'ingresso in questa modalità (dopo debounce di LUCI TMP), il relè viene attivato: `Relay=ON`.
Parte un timer per la durata dell'accensione temporizzata (`temp_duration, sec`). Questo è rappresentato dallo stato `Wait` all'interno del blocco `EllapsedTime` nel diagramma, con una transizione `after(temp_duration, sec)`.
Durante questo periodo, le luci sono accese.

Al termine di `temp_duration, sec` (transizione `after(temp_duration, sec)` dallo stato `Wait` interno a `EllapsedTime`), il sistema esegue l'azione di spegnere le luci: `Relay=OFF`. Il sistema ritorna quindi allo stato OFF della sotto-macchina "`Operation`".

Interruzione Anticipata della Temporizzazione (tramite pulsante LUCI):
Qualora il sistema è nello stato `Temp` e il pulsante LUCI viene premuto, il relè viene impostato su `Relay=OFF`, la modalità temporizzata viene interrotta e il sistema torna allo stato OFF della sotto-macchina "`Operation`".

Pressione di LUCI TMP durante la temporizzazione:
Qualora il sistema è nello stato `Temp` e il pulsante LUCI TEMP viene premuto, la pressione non ha alcun effetto.

Debounce

Una volta che il pulsante è stato rilasciato, si attiva un timer di `debounce`. Per la durata di questo timer, ulteriori cambi di stato del pulsante vengono ignorati. Se al termine del debounce lo stato del pulsante è quello atteso, allora la transizione di stato della logica principale avviene.

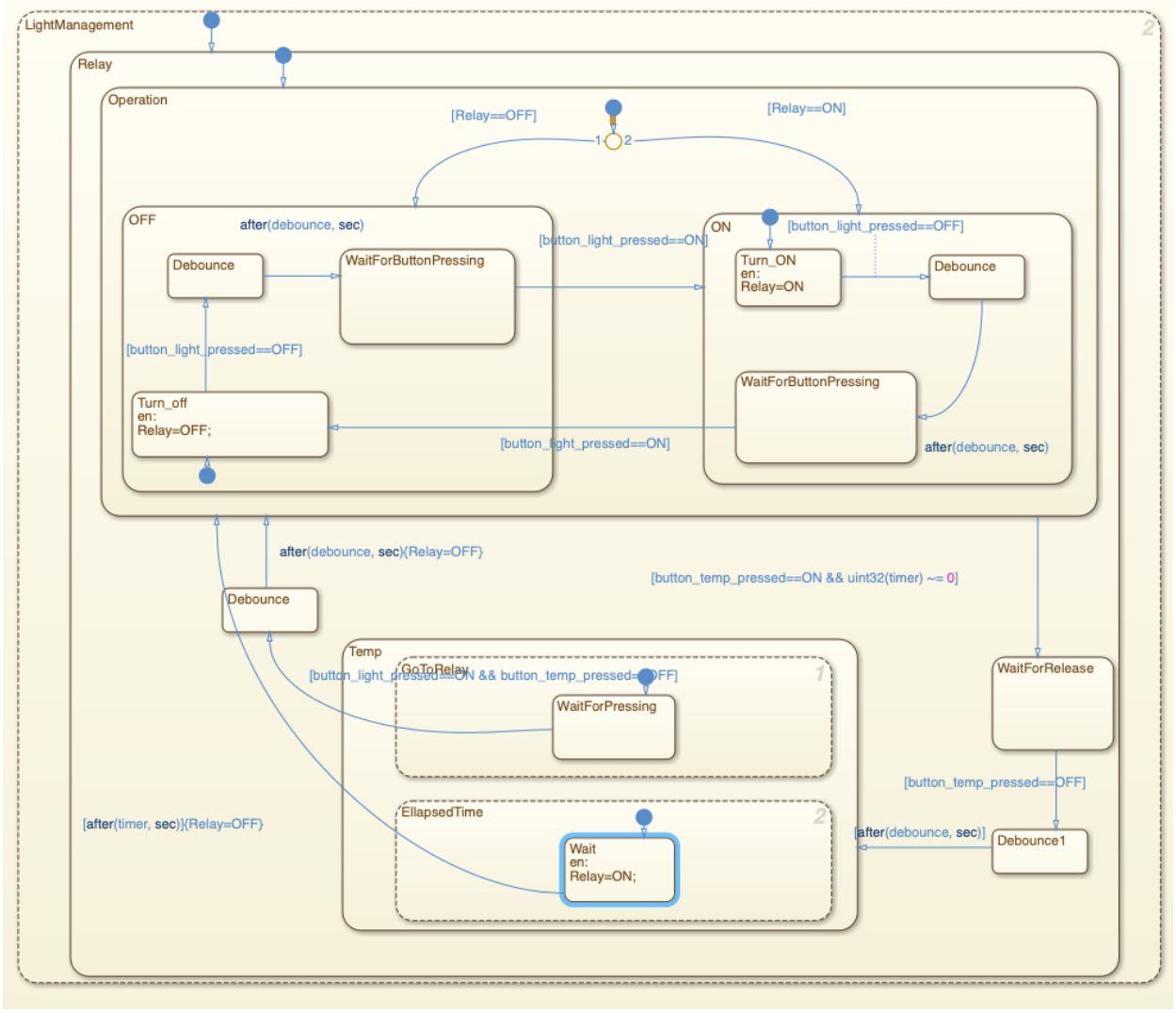


Figura 7- LightManagement

2.4. VBlindsManagement

Il modulo `VBlindsManagement` regola l'apertura delle veneziane. La sua responsabilità primaria è tradurre l'angolo di apertura desiderato, specificato dall'utente, in un segnale di controllo appropriato per il servomotore. Quest'ultimo, infatti, opera sulla base di segnali a modulazione di larghezza d'impulso (PWM).
All'interno di questo modulo, il processo inizia con la ricezione dell'angolo target per le veneziane, un valore espresso in gradi che può variare da 0° (veneziane completamente chiuse) a 180° (veneziane completamente aperte), con incrementi discreti di 20°. Questo angolo `vblinds_angle` rappresenta l'input.
Per pilotare correttamente il servomotore, è necessario convertire questo valore angolare in una percentuale di duty cycle per il segnale PWM. La logica implementata nel modulo `VBlindsManagement` utilizza una formula specifica per questa trasformazione:

$$servo_pwm_per \; = \; 5.0 + \frac{vblinds_angle}{180} * 20.0$$

Il valore calcolato `servo_pwm_per` viene quindi utilizzato per configurare il generatore PWM del microcontrollore. Il timer hardware produrrà un segnale PWM con il duty cycle percentuale determinato, inducendo il servomotore a muoversi e a mantenere la posizione angolare corrispondente all'angolo delle veneziane richiesto.

Il modulo `VBlindsManagement` regola l'apertura delle veneziane. La sua responsabilità primaria è tradurre l'angolo di apertura desiderato, specificato dall'utente, in un segnale di controllo appropriato per il servomotore. Quest'ultimo, infatti, opera sulla base di segnali a modulazione di larghezza d'impulso (PWM).
All'interno di questo modulo, il processo inizia con la ricezione dell'angolo target per le veneziane, un valore espresso in gradi `vblinds_angle` che può variare da 0° (veneziane completamente chiuse) a 180° (veneziane completamente aperte), con incrementi discreti di 20°.
Per generare il segnale PWM è stato configurato il timer general-purpose **TIM4** sul canale 1, associato al pin **PB6**. La configurazione hardware del timer è stata studiata per soddisfare i requisiti standard dei servomotori, che operano tipicamente a una frequenza di 50 Hz (periodo 20 ms).
I parametri fondamentali del timer sono stati impostati come segue:
- **Frequenza di Clock del Timer:** Dal diagramma di clock del microcontrollore, si evince che i timer sul bus **APB1**, incluso **TIM4**, sono alimentati da un clock di 16 MHz.
- **Prescaler (PSC):** Impostato al valore 1599. Il contatore del timer viene quindi incrementato a una frequenza di $\frac{16\ MHz}{(1599+1)} = 10kHz$.
- **Counter Period (ARR - Auto-Reload Register):** Impostato al valore 199. Il contatore conta da 0 a 199, definendo una risoluzione di 200 passi.

La combinazione di questi parametri determina la frequenza del segnale PWM risultante:

$$Frequenza_PWM = \frac{Frequenza_Clock}{(PSC + 1) * (ARR + 1)} = \frac{16\ 000\ 000}{(1599 + 1) * (199 + 1)} = 50Hz$$

Questa configurazione genera un segnale con un periodo esatto di **20 ms**, perfettamente adeguato al controllo del servo.
La posizione angolare del servo è determinata dalla larghezza dell'impulso alto all'interno di questi 20 ms.
Il valore numerico che definisce tale larghezza, da caricare nel registro Pulse (CCR1) del timer, è calcolato dinamicamente dal blocco Stateflow `SetAngle` tramite la seguente formula:

$$servo_pwm_out = 5.0 + \frac{vblinds_angle}{180.0} * 20.0$$

Questa equazione mappa l'intervallo angolare [0°, 180°] a un valore numerico per il registro Pulse nell'intervallo [5, 25].
Analizzando i casi limite:

- **Angolo 0°:** $servo_pwm_out = 5$. La larghezza dell'impulso è $\frac{5}{200} * 20ms = 0.5\ ms$
- **Angolo 180°:** $servo_pwm_out = 25$. La larghezza dell'impulso è $\frac{25}{200} * 20ms = 2.5\ ms$

La gamma di impulsi generata ($0.5\ ms - 2.5\ ms$) copre l'intero range operativo del servomotore.



Figura 8 - VBlindsManagement

2.5. NotifyUART

Il modulo `NotifyUART` ha il compito di comunicare periodicamente lo stato operativo del sistema di domotica al mondo esterno attraverso l'interfaccia seriale UART. Questa notifica avviene quando il dispositivo è attivo e non si trova in modalità di configurazione.
Il funzionamento di questo modulo è orchestrato da una semplice macchina a stati che cicla tra due fasi principali: la preparazione e trasmissione del messaggio "`Invia`" e un periodo di attesa "`Wait`".

Fase di "Invia":
All'ingresso in questo stato, viene chiamata la funzione `messageForUART`. Questa funzione aggrega i valori correnti delle principali variabili di stato del sistema e formattarli in un array di byte pronto per la trasmissione seriale.
Gli input specificati per `messageForUART` sono:

- ***Relay:*** Lo stato attuale del relè delle luci [*0*;*1*].
- ***vblinds_angle:*** L'angolo corrente delle veneziane [*0*,*180*].
- ***Heating_R, Heating_G, Heating_B:*** I valori componenti (Rosso, Verde, Blu) che determinano il colore del LED RGB [*0*, *255*].

All'interno di `messageForUART`, avviene la gestione rigorosamente statica della memoria per il buffer del messaggio. Viene predefinita una costante `MAX_LEN` (250 byte), che stabilisce la dimensione massima del buffer. Viene poi allocato staticamente un array di `uint8` di questa dimensione fissa, denominato `msg`. Questo approccio evita l'uso di allocazione dinamica (come `malloc`).
Per convertire i valori numerici nella loro rappresentazione testuale ASCII, viene utilizzata la funzione `write_uint_decimal`. Anche questa funzione opera con buffer statici temporanei e converte un intero in una sequenza di caratteri ASCII rappresentanti il numero in base decimale, scrivendoli direttamente nel buffer `msg`. Un indice `idx` tiene traccia della posizione corrente all'interno del buffer `msg` durante la sua costruzione.
L'output della funzione `messageForUART` è il buffer msg stesso. Sebbene `msg` abbia una dimensione fissa (`MAX_LEN`), solo una porzione iniziale di esso contiene il messaggio effettivo. La lunghezza utile del messaggio è determinata dal valore finale dell'indice `idx`.
Il firmware che gestisce la trasmissione UART deve essere consapevole di questa lunghezza effettiva per inviare solo i byte pertinenti e non l'intero buffer.
Una volta che il buffer bytes (che corrisponde a `msg`) è stato popolato, il suo contenuto viene passato al driver hardware della UART per la trasmissione seriale fisica.

Fase di "Wait":
Dopo che il messaggio è stato preparato, il modulo transita nello stato "`Wait`". In questo stato, il sistema attende per un periodo di tempo pari a 10 secondi. Durante questo periodo, il modulo `NotifyUART` non esegue altre azioni. Scaduto il timer di 10 secondi, il sistema ritorna automaticamente allo stato "`Invia`", ricominciando il ciclo di preparazione e trasmissione del messaggio di stato aggiornato.

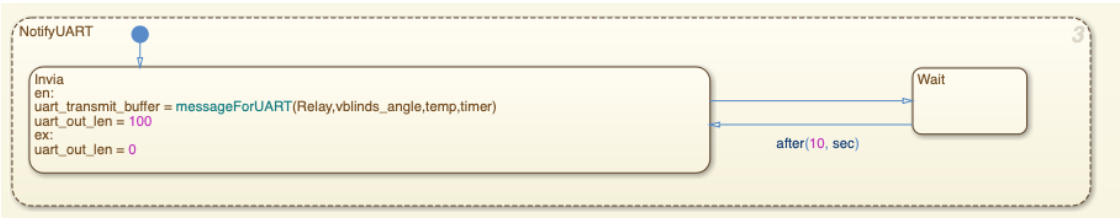


Figura 9 - NotifyUART

2.6. HeatingManagement

Il modulo `HeatingManagement` gestisce l'indicazione visiva della temperatura impostata attraverso un LED RGB. Questo modulo traduce l'attuale valore di temperatura `temp` in specifici valori di intensità per le componenti Rossa (R), Verde (G) e Blu (B) del LED, permettendo all'utente di percepire immediatamente il range di temperatura selezionato attraverso un feedback cromatico.
Per realizzare questa funzionalità, sono stati utilizzati tre canali PWM del timer **TIM3**. Come indicato in ***Tabella 3***, i canali sono stati associati ai seguenti pin:

- **TIM3 CH1** (pin **PA6**) per il LED **Rosso**;
- **TIM3 CH2** (pin **PC7**) per il LED **Verde**;
- **TIM3 CH4** (pin **PC9**) per il LED **Blu**.

La configurazione del timer **TIM3** è stata meticolosamente studiata per permettere un controllo intuitivo della luminosità. L'obiettivo era mappare direttamente il range logico a 8-bit (0-255) al comportamento fisico del LED, dove il valore 255 (usato per indicare la minima intensità di un colore) deve corrispondere allo spegnimento completo.
Per raggiungere questo scopo, mantenendo un clock di sistema di 16 MHz, i parametri del timer sono stati impostati come segue:

- **Counter Period (ARR - Auto-Reload Register):** 254. Questa scelta fa sì che il contatore del timer operi su un totale di 255 passi (da 0 a 254).
- **Prescaler (PSC):** 240.

Questa combinazione garantisce che un valore Pulse di 255 sia superiore al massimo valore raggiungibile dal contatore **CNT**, forzando il duty cycle al 100% e spegnendo così completamente il LED. La frequenza PWM risultante è di circa 260 Hz, ampiamente sufficiente a eliminare qualsiasi sfarfallio percepibile:

$$Frequenza_PWM = \frac{16\,000\,000}{(240+1)*(254+1)} = 260.23Hz$$

La logica di controllo assegna i valori da 0 a 255 alle variabili dei colori. Questi valori vengono poi usati direttamente per impostare i registri Pulse (**CCR**) dei canali PWM, ottenendo una mappatura 1:1 tra il valore logico e i 256 livelli di intensità disponibili, da massima luminosità a spegnimento totale, senza la necessità di gestire casi speciali nel software.
Il funzionamento del modulo è rappresentato da una macchina a stati dove ogni stato corrisponde a un diverso intervallo di temperatura e, di conseguenza, a una diversa combinazione di colori per il LED RGB.

Stato range_1 (Temperatura: 16°C - 18°C):

$$Heating_R = 0$$
$$Heating_G = 0$$
$$Heating_B = 255$$

In questo stato, che corrisponde alle temperature più basse del range operativo (da 16°C a 18°C, il LED RGB viene impostato per emettere luce **Blu**. Le componenti Rossa e Verde sono spente (valore 0), mentre la componente Blu è alla massima intensità (valore 255, assumendo una scala 0-255 per i valori PWM dei LED).
Se la temperatura `temp` supera i 18°C, il sistema transita allo stato range_2.

Stato range_2 (Temperatura: 19°C - 21°C):

$Heating_R = 128$
$Heating_G = 0$
$Heating_B = 128$

Questo stato corrisponde all'intervallo di temperatura da 19°C a 21°C. Il LED RGB viene impostato per emettere luce **Viola**.
Se la temperatura `temp` scende al di sotto dei 19°C, il sistema ritorna allo stato range_1.
Se la temperatura `temp` supera i 21°C, il sistema transita allo stato range_3.

Stato range_3 (Temperatura: 22°C - 24°C):

$Heating_R = 255$
$Heating_G = 255$
$Heating_B = 0$

Questo stato è attivo per temperature da 22°C a 24°C. Il LED RGB viene impostato per emettere luce **Gialla**.
Se la temperatura `temp` scende al di sotto dei 22°C, il sistema ritorna allo stato range_2.
Se la temperatura `temp` supera i 24°C, il sistema transita allo stato range_4.

Stato range_4 (Temperatura: 25°C - 27°C):

$Heating_R = 255$
$Heating_G = 128$
$Heating_B = 0$

Questo stato è per temperature da 25°C a 27°C secondo le specifiche testuali. Il LED RGB viene impostato per emettere luce **Arancione**.
Se la temperatura `temp` scende al di sotto dei 25°C, il sistema ritorna allo stato range_3.
Se la temperatura `temp` supera i 27°C, il sistema transita allo stato range_5.

Stato range_5 (Temperatura: 28°C - 30°C):

$Heating_R = 255$
$Heating_G = 0$
$Heating_B = 0$

Questo stato è per le temperature più alte, da 28°C a 30°C. Il LED RGB viene impostato per emettere luce **Rossa**.
Se la temperatura `temp` scende al di sotto di 28°C, il sistema transita allo stato range_4.

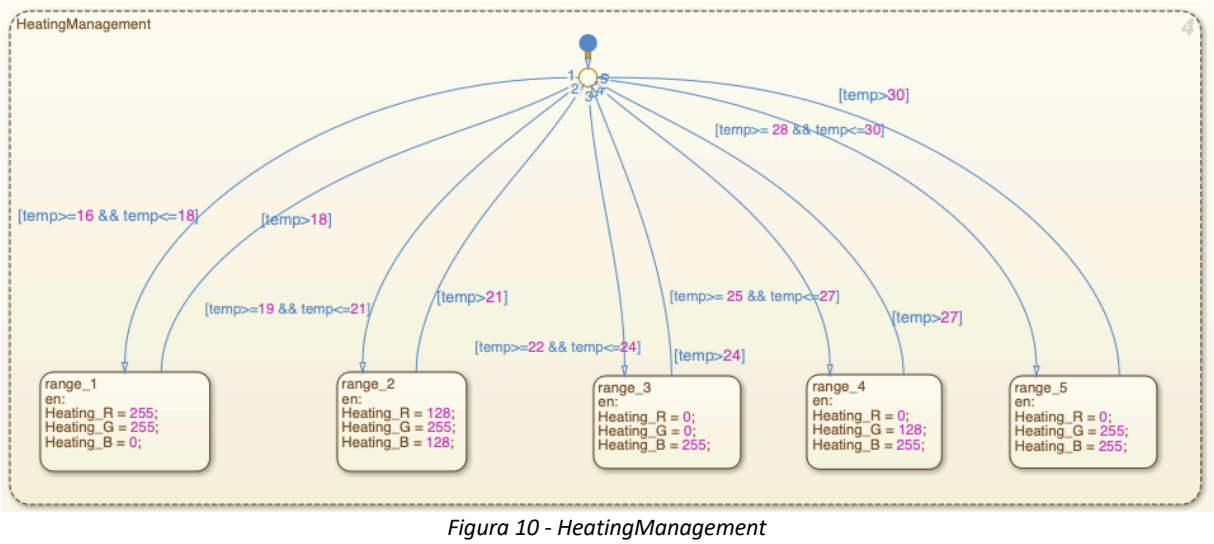


Figura 10 - HeatingManagement

2.7. BlinkingManagement

Il modulo `BlinkingManagement` genera segnali di lampeggio visivo. Il LED lampeggia a 2Hz quando il dispositivo è attivo. La logica è implementata attraverso una semplice macchina a stati con due stati distinti: "ON" e "OFF", che rappresentano rispettivamente lo stato acceso e spento del LED controllato.

Stato "ON":

Quando il modulo transita nello stato "ON", viene impostata una variabile denominata `Blinking` al valore ON (1). Questo ha l'effetto immediato di accendere il LED associato.

Il sistema rimane nello stato "ON" per $\frac{1}{2*blinking_hz}$ secondi. La transizione allo stato "OFF" avviene dopo $\frac{1}{2*blinking_hz}$ secondi, dove `blinking_hz` rappresenta la frequenza di lampeggio desiderata, espressa in Hertz.

Stato "OFF":

Analogamente, quando il modulo transita nello stato "OFF", viene impostata la variabile `Blinking` al valore OFF (0). Questo spegne il LED.

Il sistema rimane nello stato "OFF" per la stessa durata precedente. La transizione per tornare allo stato "ON" avviene dopo $\frac{1}{2*blinking_hz}$ secondi.

Il sistema alterna continuamente tra lo stato "ON" e lo stato "OFF", con ciascuno stato che dura $\frac{1}{2*blinking_hz}$ secondi. Il periodo totale T di un ciclo completo di lampeggio (ON + OFF) è quindi:

$$T = ON + OFF = \frac{1}{2 * blinking_hz} + \frac{1}{2 * blinking_hz} = \frac{1}{blinking_hz}\ secondi$$

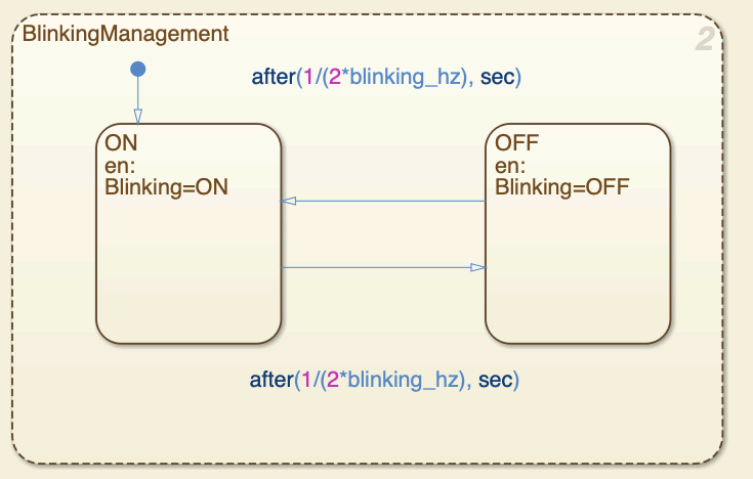


Figura 11 - BlinkingManagement

CAPITOLO III – ESECUZIONE PROGETTO

Il progetto che abbiamo realizzato va eseguito seguendo queste determinate istruzioni:

Una volta estratto l'archivio `.zip`, si troverà, oltre alla documentazione del progetto stesso, il file `model.slx` e la cartella ioc contentente il file di configurazione del target hardware.

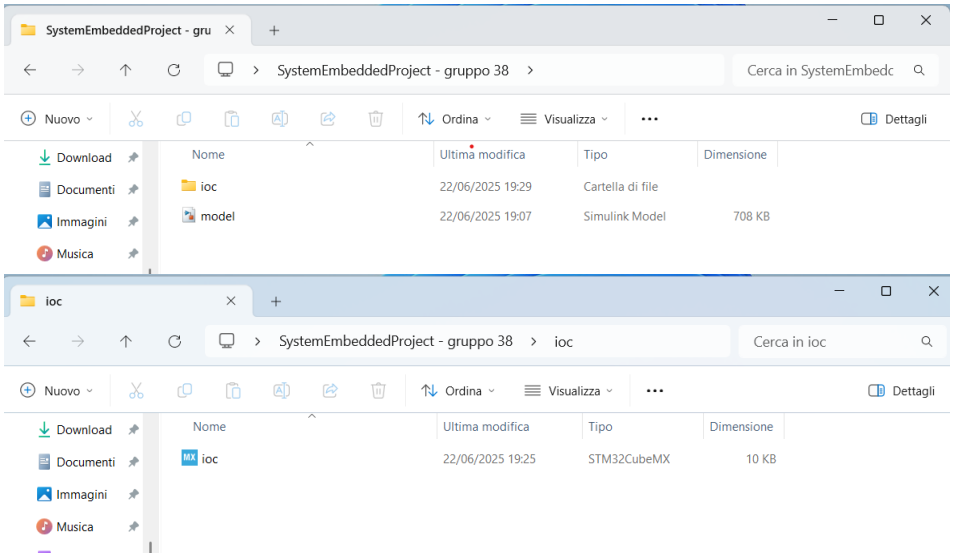


Figura 12 - Folder Content

All'apertura del file Simulink, sarà necessario aprire "`Hardware Settings`" e, selezionando "`Browse…`", scegliere il file di configurazione `.ioc`.

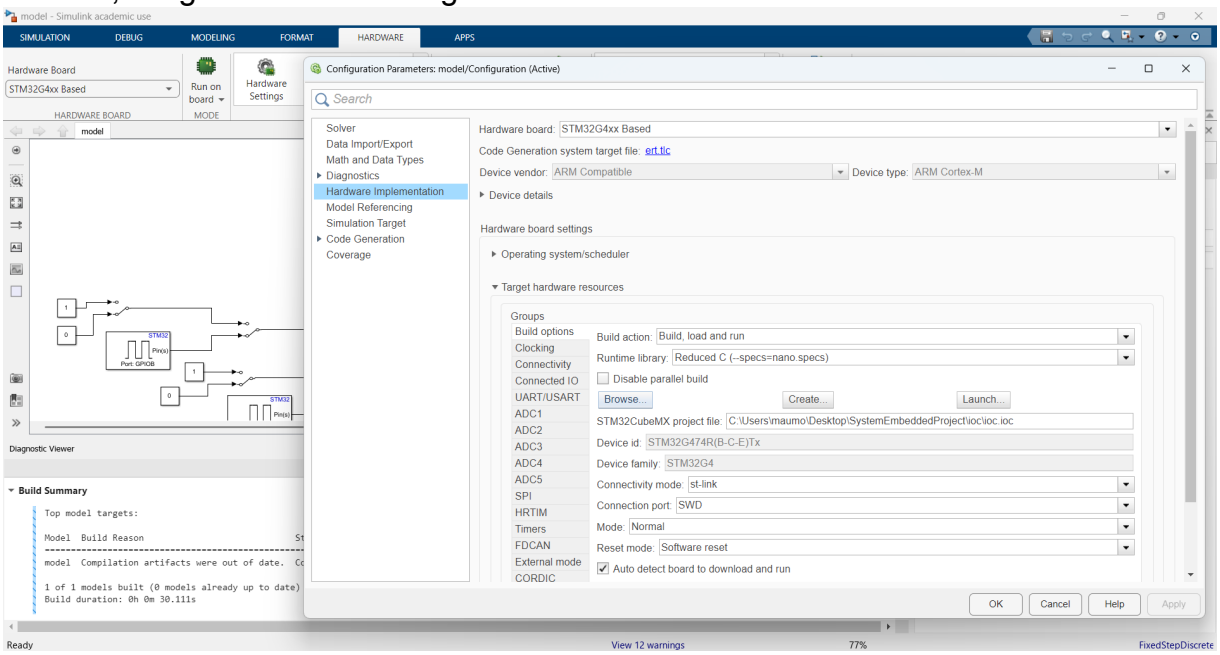


Figura 13 - Hardware Settings

In conclusione, bisogna selezionare “Build, Deploy & Start” per mettere su scheda l’eseguibile. Basterà, una volta effettuata la compilazione, aprire PuTTY e settare i parametri citati nella sezione **1.7**.

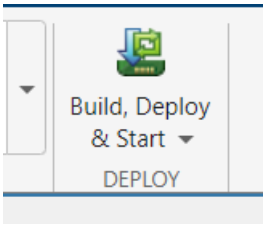


Figura 14 - Build, Deploy & Start Icon

La console PuTTY può risultare vuota, se la si apre dopo la stampa del messaggio “**[Inactive State]**”. Si consiglia di premere il pulsante di riavvio sulla scheda.

INDICE DEI CODICI, DELLE TABELLE E DELLE FIGURE

Code 1 - Stato di Configurazione (valori non settati) 2
Code 2 - Stato di Configurazione (valori settati) 2

Tabella 1 – Visualizzazione della Temperatura con LED RGB 3
Tabella 2 - Analisi dei Requisiti 4
Tabella 3 – setup dei PIN 5

Figura 1 - Stato Configuration 3
Figura 2 - Configurazione PuTTY 6
Figura 3 – Schermate di utilizzo con PuTTY 6
Figura 4 - Modello Stateflow della macchina a stati finiti per la gestione del sistema 8
Figura 5 - Concorrenza nello stato Attivo 9
Figura 6 - Concorrenza nello stato Operational 10
Figura 7- LightManagement 12
Figura 8 - VBlindsManagement 14
Figura 9 - NotifyUART 15
Figura 10 - HeatingManagement 17
Figura 11 - BlinkingManagement 18
Figura 12 - Folder Content 19
Figura 13 - Hardware Settings 19
Figura 14 - Build, Deploy & Start Icon 20